UNIVERSIDADE PAULISTA – UNIP
Projeto Integrado Multidisciplinar
Curso Superior de Tecnologia em
Análise e Desenvolvimento de Sistemas

GABRIEL SOUZA MACIEL – R413020
JULIA PENELUPPI DE FARIA – R952355
VITOR GABRIEL PEREIRA GERALDO – R8664A1

APRENDER MAIS

Solução tecnológica integrada web, mobile e desktop para uma plataforma educacional de tecnologia

São José dos Campos
2026

GABRIEL SOUZA MACIEL – R413020
JULIA PENELUPPI DE FARIA – R952355
VITOR GABRIEL PEREIRA GERALDO – R8664A1

APRENDER MAIS

Solução tecnológica integrada web, mobile e desktop para uma plataforma educacional de tecnologia

Projeto Integrado Multidisciplinar para obtenção do título de tecnólogo em Análise e Desenvolvimento de Sistemas, apresentado à Universidade Paulista – UNIP.

Orientador(a):
_____

São José dos Campos
2026

RESUMO

Este trabalho apresenta o desenvolvimento da plataforma educacional Aprender Mais, continuação do Projeto Integrado Multidisciplinar III, no qual foram realizados o levantamento de requisitos e a modelagem do sistema. A organização estudada é uma escola de cursos livres de tecnologia que controlava matrículas, entregas de atividades, notas e certificados por meio de planilhas e mensagens de e-mail, o que gerava retrabalho, falta de acompanhamento do aluno e certificados sem possibilidade de validação. O objetivo foi construir uma solução tecnológica integrada, composta por uma API REST em C# com ASP.NET Core, um site em React, um aplicativo móvel em React Native com Expo e um sistema desktop em Windows Forms, todos apoiados em um banco de dados Microsoft SQL Server com stored procedures, triggers e views. A metodologia adotada foi o Scrum, com doze sprints semanais entre setembro e novembro de 2026, quadro Kanban e versionamento no GitHub com integração contínua. A arquitetura foi organizada em camadas (Domínio, Aplicação, Infraestrutura e Apresentação), aplicando os princípios da orientação a objetos, como herança, encapsulamento, polimorfismo e uso de interfaces, o que permitiu reaproveitar as mesmas regras de negócio nas três versões. Como resultados, o sistema automatiza o cálculo do progresso do aluno, emite certificados com código de validação público, funciona no celular mesmo sem conexão com a internet e oferece recursos de acessibilidade, como ajuste do tamanho do texto e modo de alto contraste. Também foi elaborado um plano de implantação na nuvem Microsoft Azure com contêineres Docker e pipeline de CI/CD. Conclui-se que a solução atende às necessidades da organização e integra os conteúdos das disciplinas do quarto semestre em um produto funcional.

Palavras-chave: Plataforma educacional. API REST. Desenvolvimento web e mobile. Banco de dados SQL Server. Metodologia Scrum.

ABSTRACT

This work presents the development of the educational platform Aprender Mais, a continuation of the Integrated Multidisciplinary Project III, in which the system requirements and models were defined. The organization studied is a technology course school that managed enrollments, assignment submissions, grades and certificates using spreadsheets and e-mail, which caused rework, lack of student follow-up and certificates that could not be validated. The goal was to build an integrated technological solution composed of a REST API written in C# with ASP.NET Core, a React website, a mobile application in React Native with Expo and a Windows Forms desktop system, all backed by a Microsoft SQL Server database with stored procedures, triggers and views. The Scrum methodology was adopted, with twelve weekly sprints between September and November 2026, a Kanban board and version control on GitHub with continuous integration. The architecture was organized in layers (Domain, Application, Infrastructure and Presentation), applying object-oriented principles such as inheritance, encapsulation, polymorphism and interfaces, which allowed the same business rules to be reused in the three versions. As a result, the system automatically calculates student progress, issues certificates with a public validation code, works on mobile phones even without an internet connection and offers accessibility features such as text size adjustment and high contrast mode. A deployment plan for the Microsoft Azure cloud with Docker containers and a CI/CD pipeline was also prepared. It is concluded that the solution meets the needs of the organization and integrates the subjects of the fourth semester into a functional product.

Keywords: Educational platform. REST API. Web and mobile development. SQL Server database. Scrum methodology.

LISTA DE ABREVIATURAS E SIGLAS

API	Application Programming Interface (Interface de Programação de Aplicações)
CI/CD	Continuous Integration / Continuous Delivery (Integração e Entrega Contínuas)
CORS	Cross-Origin Resource Sharing (Compartilhamento de Recursos entre Origens)
DTO	Data Transfer Object (Objeto de Transferência de Dados)
HTTP	Hypertext Transfer Protocol (Protocolo de Transferência de Hipertexto)
JSON	JavaScript Object Notation
LGPD	Lei Geral de Proteção de Dados Pessoais
MER	Modelo Entidade-Relacionamento
POO	Programação Orientada a Objetos
REST	Representational State Transfer
SQL	Structured Query Language (Linguagem de Consulta Estruturada)
UC	Use Case (Caso de Uso)
UML	Unified Modeling Language (Linguagem de Modelagem Unificada)
WCAG	Web Content Accessibility Guidelines (Diretrizes de Acessibilidade para Conteúdo Web)

SUMÁRIO

1 INTRODUÇÃO.....7
2 CARACTERIZAÇÃO DA ORGANIZAÇÃO.....9
2.1 Segmento de atuação, produtos e serviços.....9
2.2 Estrutura organizacional e principais processos.....9
2.3 Problemas identificados e oportunidades de melhoria.....10
3 PLANEJAMENTO DA SOLUÇÃO TECNOLÓGICA.....12
3.1 Problema, objetivos e público-alvo.....12
3.2 Proposta de valor e modelo de negócio.....12
3.3 Análise de viabilidade.....13
3.4 Benefícios esperados e diferenciais competitivos.....13
4 RESPONSABILIDADE SOCIAL E DIVERSIDADE.....15
4.1 Combate à discriminação e valorização da diversidade.....15
4.2 Recursos tecnológicos inclusivos e acessibilidade.....15
4.3 Tecnologia para promoção da cidadania.....16
5 ARQUITETURA DE SOFTWARE.....17
5.1 Organização em camadas.....17
5.2 Orientação a objetos.....18
5.3 Modularização e reaproveitamento.....19
5.4 Versão desktop.....19
6 PROJETO DO BANCO DE DADOS.....20
6.1 Modelo entidade-relacionamento e modelo lógico.....20
6.2 Modelo físico.....21
6.3 Views, procedures e triggers.....22
6.4 Script completo.....23
7 DESENVOLVIMENTO DA SOLUÇÃO WEB.....24
7.1 Arquitetura da aplicação.....24
7.2 APIs REST.....24
7.3 Autenticação.....25
7.4 Segurança.....25
7.5 Estrutura administrativa e integração entre módulos.....25
8 DESENVOLVIMENTO DA SOLUÇÃO MOBILE.....28
8.1 Telas principais.....28
8.2 Fluxo de navegação e autenticação.....29
8.3 Sincronização de dados.....30
8.4 Integração com APIs.....31
9 INFRAESTRUTURA EM NUVEM E DEVOPS.....33
9.1 Arquitetura em nuvem e serviços utilizados.....33

9.2 Contêineres e pipeline CI/CD.....34
9.3 Monitoramento, escalabilidade e segurança.....34
10 GERENCIAMENTO ÁGIL DO PROJETO.....35
10.1 Product Backlog.....35
10.2 Sprint Backlog e cronograma das sprints.....36
10.3 Quadro Kanban e definição das entregas.....37
11 CONCLUSÃO.....38
REFERÊNCIAS.....39
APÊNDICE A – REPOSITÓRIO E ACESSO AO SISTEMA.....40

1 INTRODUÇÃO

A transformação digital tornou-se um fator de competitividade para organizações de todos os segmentos, inclusive para as instituições de ensino. Escolas que ainda dependem de planilhas, mensagens de e-mail e documentos produzidos manualmente encontram dificuldade para acompanhar seus alunos, garantir a confiabilidade das informações e oferecer uma experiência adequada a um público que estuda cada vez mais pelo celular.

No Projeto Integrado Multidisciplinar III, a equipe estudou a escola de cursos livres de tecnologia Aprender Mais, levantou os requisitos funcionais e não funcionais e produziu a modelagem do sistema, com diagramas de casos de uso, de classes, de sequência e o modelo do banco de dados. Este trabalho dá continuidade a esse estudo: o objetivo é **desenvolver e planejar a implantação da solução tecnológica completa**, contemplando aplicações web, mobile e desktop integradas por uma API, um banco de dados corporativo, a arquitetura de software, a infraestrutura em nuvem e o gerenciamento ágil do projeto.

Como objetivos específicos, buscou-se: caracterizar a organização e seus processos; planejar a solução e analisar a viabilidade do modelo de negócio; incorporar ações de inclusão, diversidade e acessibilidade; construir a API REST e as aplicações web e mobile; definir a arquitetura em camadas orientada a objetos; implementar o banco de dados com procedures e triggers; elaborar o plano de nuvem e DevOps; e organizar o desenvolvimento com Scrum.

A metodologia utilizada foi a pesquisa aplicada, com abordagem prática e colaborativa. O desenvolvimento seguiu o framework Scrum (SCHWABER; SUTHERLAND, 2020), em sprints semanais de setembro a novembro de 2026, com o código-fonte versionado no GitHub e verificado automaticamente por integração contínua. As decisões técnicas foram fundamentadas na literatura de engenharia de software (SOMMERVILLE, 2018; PRESSMAN; MAXIM, 2016), de arquitetura (MARTIN, 2019; FOWLER, 2006) e de banco de dados (ELMASRI; NAVATHE, 2018), além da documentação oficial das tecnologias empregadas.

O trabalho está organizado conforme as etapas do manual do PIM IV: a seção 2 caracteriza a organização; a seção 3 apresenta o planejamento da solução; a seção 4 trata de responsabilidade social e diversidade; a seção 5 descreve a arquitetura de

software; a seção 6, o banco de dados; as seções 7 e 8, as soluções web e mobile; a seção 9, a infraestrutura em nuvem e DevOps; e a seção 10, o gerenciamento ágil. Por fim, a seção 11 apresenta a conclusão.

2 CARACTERIZAÇÃO DA ORGANIZAÇÃO

A Aprender Mais – Educação Tecnológica é uma escola de cursos livres de tecnologia da informação com sede em São José dos Campos (SP), modelada no PIM III. Ela atende pessoas de qualquer faixa etária que desejam ingressar ou crescer na área de tecnologia, de forma presencial e remota.

2.1 Segmento de atuação, produtos e serviços

A organização atua no segmento de educação, no ramo de cursos livres de tecnologia para iniciantes e estudantes de nível intermediário. O Quadro 1 apresenta os cursos oferecidos. Cada curso possui atividades avaliativas corrigidas pela secretaria acadêmica, e quem conclui todas as atividades recebe um certificado de conclusão.

Quadro 1 – Cursos oferecidos pela Aprender Mais

Curso	**Categoria**	**Nível**	**Carga horária**
Engenharia de Software	Engenharia	Iniciante	18 h (28 aulas)
UX/UI Design	Design	Intermediário	14 h (24 aulas)
Microsoft 365	Produtividade	Iniciante	10 h (18 aulas)
Fundamentos de T.I.	Base técnica	Iniciante	12 h (21 aulas), em reformulação

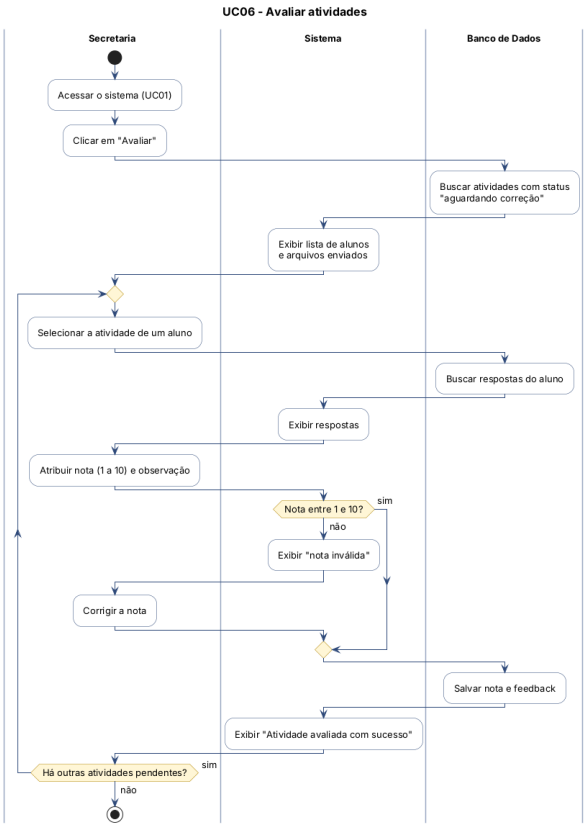
Fonte: elaborado pelos autores (2026).

2.2 Estrutura organizacional e principais processos

A estrutura é composta pela direção, responsável também pela administração do sistema; pela secretaria acadêmica, que cuida de matrículas, atividades, correções e certificados; pelos conteudistas e tutores, que produzem as aulas; pelo suporte de tecnologia; e pelos alunos. Os três perfis do sistema refletem essa estrutura: **Administrador**, **Secretaria** e **Aluno**.

Os principais processos, já representados pelos casos de uso do PIM III, são: cadastro do aluno (UC02) e acesso ao sistema (UC01); matrícula e estudo dos conteúdos (UC03); entrega das atividades (UC04); consulta de notas (UC05); correção das atividades (UC06); gestão de conteúdos (UC07 a UC09); e gestão de acessos, com cadastro de secretarias e desativação de cadastros (UC10 e UC11). No PIM IV, cada caso de uso recebeu um diagrama de atividade, que mostra o fluxo das ações e os caminhos alternativos. A Figura 1 apresenta o exemplo do UC06.

Figura 1 – Diagrama de atividade do UC06 – Avaliar atividades



Fonte: elaborado pelos autores (2026).

2.3 Problemas identificados e oportunidades de melhoria

O levantamento realizado no PIM III apontou problemas relevantes: matrículas e notas controladas em planilhas e e-mails, com retrabalho e risco de perda de dados; alunos sem visão do próprio progresso; entregas recebidas fora do prazo e sem padronização; certificados produzidos manualmente, sem forma de validação; ausência de acesso pelo celular, embora muitos alunos estudem no transporte com internet instável; ex-funcionários que mantinham acesso aos materiais; e falta de indicadores para a direção. O Quadro 2 relaciona cada problema à oportunidade de melhoria explorada pela solução.

Quadro 2 – Problemas e oportunidades de melhoria por meio da tecnologia

Problema	**Oportunidade (o que o sistema faz)**
Controle em planilhas e e-mail	Banco de dados único (SQL Server) acessado por uma API usada pelas versões web, mobile e desktop
Aluno sem visão do progresso	Progresso calculado automaticamente a cada correção e exibido em barra
Entregas desorganizadas	Envio pelo sistema com controle de prazo e fila de correção
Certificado sem validação	Emissão automática com código validado em página pública
Internet instável	Aplicativo com sincronização offline

Problema	Oportunidade (o que o sistema faz)
Acessos indevidos	Desativação de cadastros com bloqueio imediato e registro em log de auditoria
Falta de indicadores	Painel do administrador com relatórios gerados por stored procedures

Fonte: elaborado pelos autores (2026).

3 PLANEJAMENTO DA SOLUÇÃO TECNOLÓGICA

Esta etapa, ligada à disciplina de Empreendedorismo em TI, define a solução proposta e analisa a viabilidade do negócio. Segundo Dornelas (2021), o planejamento permite transformar uma ideia em oportunidade ao identificar o problema, o público e a forma de gerar valor de maneira sustentável.

3.1 Problema, objetivos e público-alvo

O problema central é a perda de tempo e de alunos causada por processos acadêmicos manuais e dispersos, somada à falta de acesso pelo celular e de visibilidade do progresso. A solução tem como objetivos: centralizar os processos em um único sistema; oferecê-lo em três plataformas sobre a mesma API; calcular o progresso e emitir certificados automaticamente, com validação pública; permitir o estudo e a resposta de atividades mesmo sem internet; fornecer indicadores à direção; e garantir acessibilidade e inclusão.

O público-alvo inclui jovens em busca do primeiro emprego em tecnologia, adultos em transição de carreira, estudantes de escola pública e de baixa renda, pessoas com deficiência visual ou baixa visão e a própria equipe da escola, que precisa de menos trabalho manual e de mais controle.

3.2 Proposta de valor e modelo de negócio

A proposta de valor é resumida na frase "aprenda tecnologia no seu ritmo, do primeiro clique ao certificado": um ambiente simples e acessível, no qual o aluno sabe exatamente quanto falta para concluir, recebe correções feitas por pessoas e ganha um certificado que qualquer empresa pode validar. O modelo de negócio foi estruturado com o Business Model Canvas (OSTERWALDER; PIGNEUR, 2011), apresentado no Quadro 3.

Quadro 3 – Business Model Canvas da Aprender Mais

Bloco	**Descrição**
Segmentos de clientes	Público-alvo descrito acima e empresas parceiras que contratam turmas
Proposta de valor	Cursos práticos, progresso visível, certificado validável, aplicativo offline e acessibilidade
Canais	Site, aplicativo, redes sociais e parcerias com escolas públicas e ONGs

Bloco	Descrição
Relacionamento	Feedback individual nas correções e suporte pela secretaria
Fontes de receita	Modelo freemium: cursos de base gratuitos e cursos completos com certificado por mensalidade; turmas para empresas (B2B)
Recursos-chave	A plataforma desenvolvida, conteudistas e secretaria acadêmica
Atividades-chave	Produção de conteúdo, correção de atividades e manutenção da plataforma
Parcerias-chave	Escolas públicas, ONGs e empresas de tecnologia do Vale do Paraíba
Estrutura de custos	Infraestrutura em nuvem, equipe pedagógica, marketing e desenvolvimento

Fonte: elaborado pelos autores (2026).

3.3 Análise de viabilidade

Do ponto de vista técnico, a solução é viável porque a equipe domina as tecnologias escolhidas, todas estudadas no curso e disponíveis gratuitamente para desenvolvimento. Do ponto de vista financeiro, o Quadro 4 apresenta uma estimativa mensal de infraestrutura com base nos planos de entrada da Microsoft Azure; os valores são aproximados e devem ser confirmados na calculadora de preços do provedor no momento da contratação.

Quadro 4 – Estimativa mensal de custos de infraestrutura

Item	Plano	Estimativa mensal
Azure App Service (API)	B1	US$ 13
Azure SQL Database	Basic / S0	US$ 5 a 15
Azure Static Web Apps (site)	Gratuito	US$ 0
Application Insights	Até 5 GB de logs	US$ 0
Total aproximado		US$ 20 a 30 (cerca de R$ 110 a 170)

Fonte: elaborado pelos autores com base em Microsoft (2026).

Em um cenário conservador, com 60 alunos pagantes e mensalidade de R$ 49,90, a receita mensal é de R$ 2.994, suficiente para cobrir a infraestrutura com folga; o ponto de equilíbrio da infraestrutura é de apenas quatro alunos pagantes. O maior custo do negócio, portanto, é a equipe pedagógica, e não a tecnologia, o que confirma a viabilidade da proposta.

3.4 Benefícios esperados e diferenciais competitivos

Os benefícios esperados são a redução do trabalho manual da secretaria, a diminuição das desistências pelo acompanhamento do progresso, certificados

confiáveis validados em segundos, o atendimento a alunos com pouca internet e a disponibilidade de dados para decisões da direção. Os diferenciais competitivos são:

- **sincronização offline** no aplicativo, que permite responder atividades sem conexão;
- **certificado com código de validação** público;
- **acessibilidade**, com fonte voltada a pessoas com baixa visão, alto contraste e ajuste do tamanho do texto;
- **correção humana com feedback**, em vez de apenas questionários automáticos;
- **três plataformas** (web, mobile e desktop) sobre as mesmas regras de negócio.

4 RESPONSABILIDADE SOCIAL E DIVERSIDADE

A tecnologia só promove transformação quando está acessível a todas as pessoas. Esta seção, relacionada à disciplina de Relações Étnico-Raciais e Afrodescendência, apresenta as ações já implementadas no sistema e as ações propostas para a operação da escola.

4.1 Combate à discriminação e valorização da diversidade

O cadastro solicita apenas nome, CPF, e-mail e senha, sem campos de raça, gênero, religião ou foto que possam induzir tratamento desigual: a secretaria corrige a resposta, não a pessoa. Prazos, notas e certificados seguem regras automáticas no código e no banco de dados, sem exceções manuais. Propõe-se ainda um termo de uso que proíba manifestações discriminatórias nos comentários de correção, com canal de denúncia pela secretaria. Os dados pessoais são tratados de forma mínima e as senhas são armazenadas com hash, em conformidade com a LGPD (BRASIL, 2018).

Para valorizar a diversidade, propõe-se incluir nos cursos casos e trajetórias de profissionais negros, indígenas e mulheres da tecnologia, em consonância com a Lei nº 10.639/2003, que tornou obrigatória a temática da história e cultura afro-brasileira na educação (BRASIL, 2003). Também se propõe um programa de bolsas, financiado por parte das mensalidades, com prioridade para estudantes de escola pública pretos, pardos e indígenas, em parceria com ONGs da região.

4.2 Recursos tecnológicos inclusivos e acessibilidade

A Lei Brasileira de Inclusão (BRASIL, 2015) garante à pessoa com deficiência o acesso à informação e à comunicação, e as diretrizes WCAG 2.1 (W3C, 2018) orientam como tornar o conteúdo web perceptível, operável, compreensível e robusto. Com base nessas referências, foram implementados os recursos do Quadro 5.

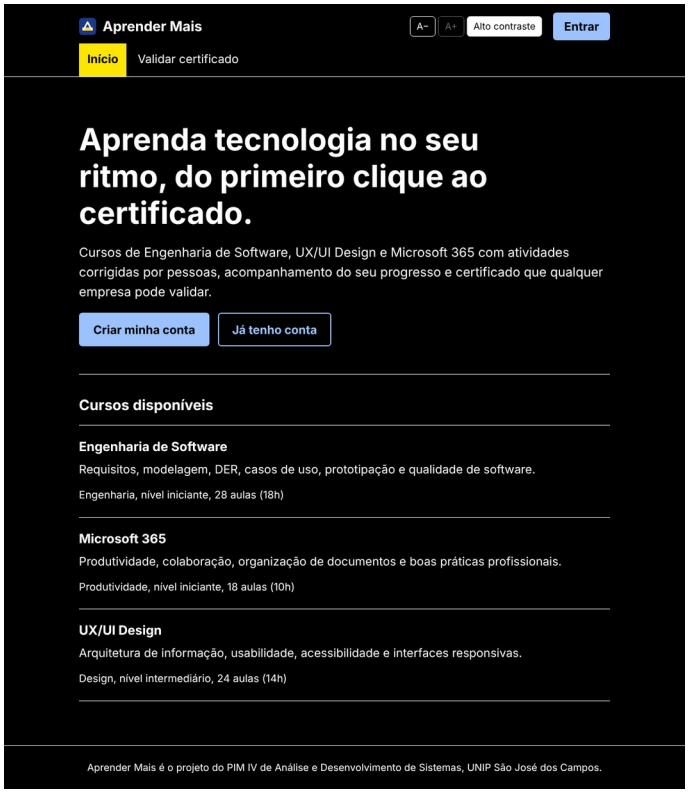
Quadro 5 – Recursos de acessibilidade implementados

Recurso	**Descrição**
Fonte Atkinson Hyperlegible	Fonte desenvolvida para pessoas com baixa visão, usada em todo o site
Ajuste do tamanho do texto	Botões A− e A+ no topo de todas as páginas, com a preferência salva no navegador
Alto contraste	Fundo preto, texto branco e destaques amarelos

Recurso	Descrição
Navegação por teclado	Link "Pular para o conteúdo" e contorno de foco visível em todos os elementos
Leitores de tela	Rótulos visíveis nos campos, atributos ARIA, tabelas com legenda e cabeçalhos
Contraste de cores	Combinações dentro do nível AA da WCAG
Redução de movimento	Respeito à preferência do sistema operacional
Aplicativo offline	Inclusão de quem tem internet limitada ou plano de dados pequeno

Fonte: elaborado pelos autores (2026).

Figura 2 – Página inicial com texto ampliado e alto contraste



Fonte: elaborado pelos autores (2026).

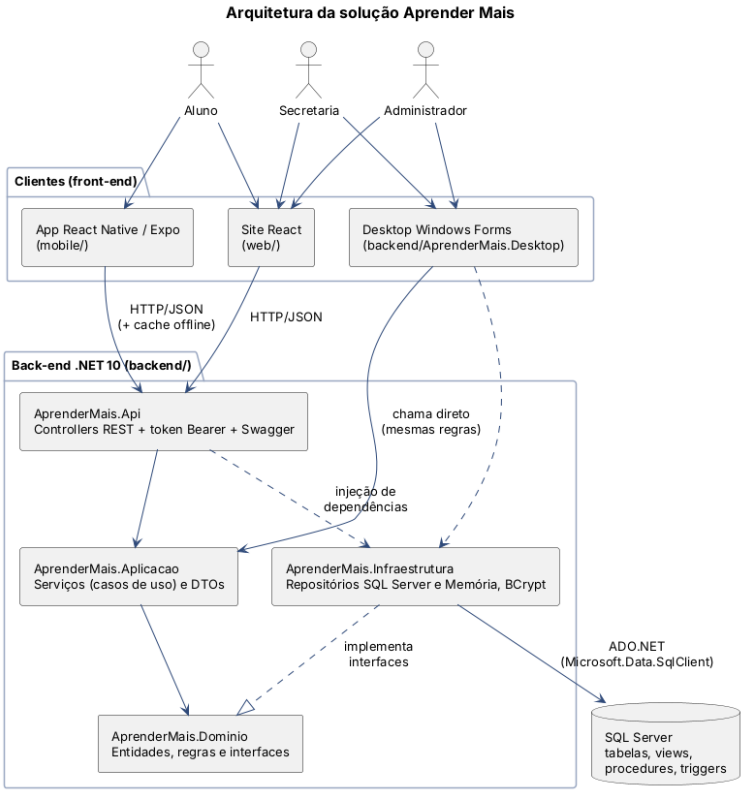
4.3 Tecnologia para promoção da cidadania

Os cursos de Microsoft 365 e Fundamentos de TI preparam para vagas administrativas e técnicas, ampliando a empregabilidade. O certificado validável permite ao aluno comprovar o que aprendeu para qualquer empresa, sem burocracia. Propõe-se ainda a realização de oficinas gratuitas de inclusão digital em escolas públicas de São José dos Campos, usando a própria plataforma. Por fim, o visitante pode consultar o catálogo de cursos sem cadastro, e a conta só é solicitada quando ele decide estudar.

5 ARQUITETURA DE SOFTWARE

A arquitetura, relacionada à disciplina de Programação Aplicada em .NET, foi definida para que as aplicações web, mobile e desktop compartilhem as mesmas regras de negócio. A Figura 3 apresenta a visão geral da solução: o site e o aplicativo comunicam-se com a API por HTTP, enquanto o sistema desktop utiliza diretamente os mesmos projetos de aplicação e domínio da API.

Figura 3 – Arquitetura da solução Aprender Mais



Fonte: elaborado pelos autores (2026).

5.1 Organização em camadas

O back-end foi escrito em C# com .NET 10 e dividido em camadas, seguindo o princípio de que as dependências apontam para dentro: as regras de negócio não conhecem a interface nem o banco de dados (MARTIN, 2019). Essa separação, descrita por Fowler (2006) como arquitetura em camadas, facilita a manutenção e permite substituir partes do sistema sem afetar as demais. O Quadro 6 descreve cada camada.

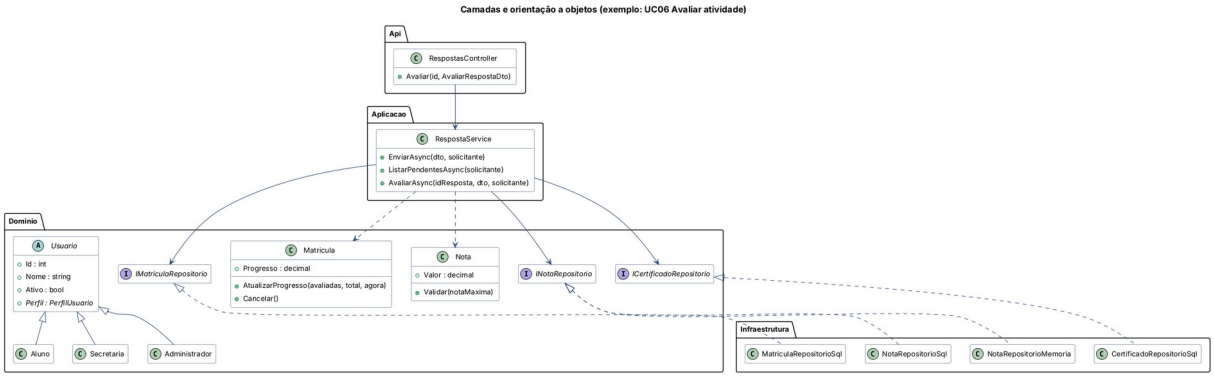
Quadro 6 – Camadas do back-end

Camada (projeto)	Responsabilidade	Exemplos
Domínio	Entidades, regras de negócio e	Usuario,

Camada (projeto)	Responsabilidade	Exemplos
(AprenderMais.Dominio)	interfaces dos repositórios	Matricula.AtualizarProgresso(), Nota.Validar(), IMatriculaRepositorio
Aplicação (AprenderMais.Aplicacao)	Casos de uso: um serviço por assunto, DTOs e verificação de permissões	RespostaService.AvaliarAsync(), ConteudoService, Permissoes
Infraestrutura (AprenderMais.Infraestrutura)	Acesso a dados (SQL Server ou memória) e segurança	MatriculaRepositorioSql, BCryptPasswordHasher
Apresentação (AprenderMais.Api e AprenderMais.Desktop)	Forma de acesso do usuário	Controllers REST e telas Windows Forms

Fonte: elaborado pelos autores (2026).

Figura 4 – Camadas e orientação a objetos no caso de uso UC06



Fonte: elaborado pelos autores (2026).

5.2 Orientação a objetos

Os conceitos de orientação a objetos (LARMAN, 2007) aparecem de forma concreta no código. Há **herança** nas classes Aluno, Secretaria e Administrador, que estendem a classe abstrata Usuario, exatamente como no diagrama de classes do PIM III. O **polimorfismo** está na propriedade abstrata Perfil, que cada subclasse implementa de forma diferente. O **encapsulamento** aparece nas regras guardadas dentro das próprias entidades: a classe Matricula impede o cancelamento de uma matrícula concluída, e a classe Nota impede nota acima da máxima. As **interfaces** de repositório definem o que é necessário para acessar os dados, sem dizer como, e a **injeção de dependência** entrega a cada serviço os repositórios de que ele precisa por meio do construtor. O Quadro 7 mostra a regra de progresso encapsulada na entidade Matricula.

Quadro 7 – Regra de progresso encapsulada na classe Matricula (C#)

```
public void AtualizarProgresso(int avaliadas, int total, DateTime agoraUtc)
{
    if (Status == StatusMatricula.Cancelado || total <= 0)
        return;

    Progresso = Math.Round(100m * avaliadas / total, 2);

    if (Progresso >= 100m)
    {
        Progresso = 100m;
        Status = StatusMatricula.Concluido;
        DataConclusao ??= agoraUtc;
    }
}
```

Fonte: código-fonte do projeto (2026).

5.3 Modularização e reaproveitamento

A solução possui cinco projetos e um projeto de testes. Uma fábrica de repositórios escolhe, conforme o arquivo de configuração, se os dados ficam em memória, para demonstração e testes, ou no SQL Server; os serviços e as telas não mudam com essa escolha. Os projetos web e mobile são independentes e se comunicam apenas pela API. A qualidade é verificada por 49 testes automatizados com xUnit, que cobrem regras de domínio e serviços, como o envio de respostas fora do prazo, a nota acima da máxima e a emissão automática do certificado ao concluir todas as atividades (SOMMERVILLE, 2018).

5.4 Versão desktop

A versão desktop foi desenvolvida em Windows Forms e reaproveita diretamente os projetos de domínio, aplicação e infraestrutura. A janela principal exibe abas conforme o perfil: o aluno vê o catálogo, as matrículas e as notas; a secretaria gerencia conteúdos e corrige atividades em uma janela própria, com nota e feedback; e o administrador lista, cadastra, ativa e desativa usuários. Como o código das regras é o mesmo da API, o comportamento é idêntico nas três versões.

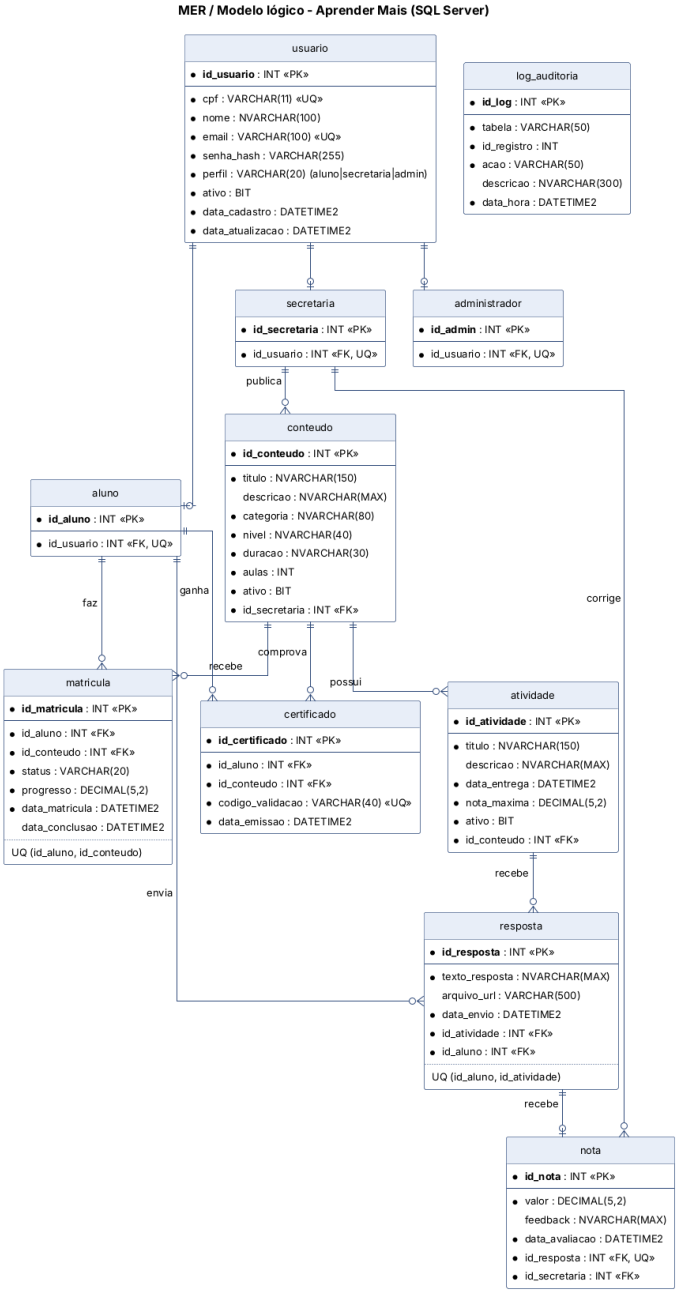
6 PROJETO DO BANCO DE DADOS

Nesta etapa, ligada à disciplina de Programação de Banco de Dados, o banco modelado em MySQL no PIM III foi migrado para o Microsoft SQL Server, pela integração com o .NET e com o Azure SQL Database. Os tipos foram adaptados: AUTO_INCREMENT foi substituído por IDENTITY, ENUM por restrições CHECK, BOOLEAN por BIT e TEXT por NVARCHAR(MAX). Também foram acrescentados views, stored procedures, triggers e uma tabela de auditoria.

6.1 Modelo entidade-relacionamento e modelo lógico

A Figura 5 apresenta o MER do PIM IV. A herança do modelo de classes foi representada por tabelas de especialização: aluno, secretaria e administrador referenciam a tabela usuario, que guarda os dados comuns. Um aluno faz matrículas em conteúdos; cada conteúdo possui atividades; o aluno envia respostas para as atividades; cada resposta recebe no máximo uma nota, dada por uma secretaria; e o certificado liga o aluno ao conteúdo concluído. O modelo está na terceira forma normal (ELMASRI; NAVATHE, 2018); o progresso da matrícula é o único dado calculado armazenado, por desempenho, e é recalculado a cada correção.

Figura 5 – Modelo entidade-relacionamento (SQL Server)



Fonte: elaborado pelos autores (2026).

6.2 Modelo físico

O modelo físico foi implementado no script de tabelas com chaves primárias IDENTITY, chaves estrangeiras e restrições nomeadas. O Quadro 8 resume os recursos utilizados.

Quadro 8 – Recursos do modelo físico

Recurso	Uso no projeto
INT IDENTITY(1,1)	Chaves primárias geradas automaticamente
NVARCHAR	Textos com acentos: nome, título, descrição

Recurso	Uso no projeto
DATETIME2(0)	Datas em UTC; o prazo de entrega fica em horário de Brasília
DECIMAL(5,2)	Notas e progresso, por exemplo 9,50 e 66,67
CHECK	Perfil, status da matrícula, progresso entre 0 e 100, quantidade de aulas não negativa
UNIQUE	E-mail e CPF; uma matrícula por aluno e curso; uma resposta por atividade; uma nota por resposta
DEFAULT	Campo ativo igual a 1 e datas com SYSUTCDATETIME()
Índices	Consultas frequentes: conteúdos ativos, atividades por conteúdo, matrículas por aluno

Fonte: elaborado pelos autores (2026).

6.3 Views, procedures e triggers

As **views** guardam consultas usadas com frequência: vw_usuario_perfil junta o usuário ao seu perfil; vw_respostas_pendentes lista as respostas ainda sem nota, que formam a fila de correção da secretaria; e vw_boletim reúne as notas com aluno, curso e atividade. As **stored procedures** concentram operações importantes e são chamadas diretamente pela API, conforme o Quadro 9.

Quadro 9 – Stored procedures

Procedure	Função	Chamada por
sp_cadastrar_usuario	Insere o usuário e o perfil na mesma transação; recusa e-mail ou CPF repetido	Cadastro do aluno (UC02) e da secretaria (UC10)
sp_calcular_progresso	Conta as atividades avaliadas do aluno e o total do curso	Correção da atividade (UC06)
sp_emitir_certificado	Emite o certificado com código de validação ou devolve o existente	Conclusão do curso
sp_relatorio_resumo	Nove indicadores gerais da plataforma	Painel do administrador
sp_relatorio_por_conteudo	Matriculados, concluintes e média por curso	Painel do administrador

Fonte: elaborado pelos autores (2026).

Os **triggers** garantem regras mesmo que os dados sejam alterados fora da aplicação: trg_usuario_auditoria registra no log a ativação e a desativação de usuários; trg_conteudo_desativado desativa as atividades de um curso desativado; trg_nota_validar impede nota negativa ou maior que a máxima; e trg_resposta_validar só aceita respostas dentro do prazo, de atividades ativas e de alunos matriculados. As regras importantes existem, portanto, nas duas pontas: no C#, para exibir mensagens amigáveis, e no banco, como última defesa da integridade dos dados. O Quadro 10 apresenta um dos triggers.

Quadro 10 – Trigger que valida a nota (T-SQL)

```
CREATE OR ALTER TRIGGER trg_nota_validar
ON nota
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF EXISTS (
        SELECT 1
          FROM inserted i
          JOIN resposta r   ON r.id_resposta   = i.id_resposta
          JOIN atividade at ON at.id_atividade = r.id_atividade
         WHERE i.valor < 0 OR i.valor > at.nota_maxima
    )
        THROW 50010,
              N'A nota deve estar entre 0 e a nota máxima da atividade.', 1;
END
```

Fonte: código-fonte do projeto (2026).

6.4 Script completo

O banco é criado por seis scripts executados em ordem: criação do banco, tabelas, views, procedures, triggers e dados iniciais de demonstração. Os seis também foram reunidos no arquivo script_completo.sql, que pode ser executado de uma só vez no SQL Server Management Studio e é o mesmo utilizado pela API, pelo site, pelo aplicativo e pelo desktop. O script está disponível na pasta database do repositório do projeto (Apêndice A).

7 DESENVOLVIMENTO DA SOLUÇÃO WEB

A solução web, relacionada à disciplina de Desenvolvimento Web com .NET, é formada por duas partes: a API REST em ASP.NET Core, que é o núcleo do sistema, e o site em React com JavaScript, que oferece a interface para os três perfis.

7.1 Arquitetura da aplicação

O site foi criado com a ferramenta Vite e a biblioteca React (REACT, 2026). Ele é organizado em páginas por perfil (público, aluno, secretaria e administrador), componentes reaproveitáveis (layout, menu, rota protegida, barra de acessibilidade e mensagens), um contexto que guarda o usuário logado e um único módulo de serviços, por onde passam todas as chamadas à API. Assim, nenhuma tela conhece endereços ou detalhes de comunicação: ela apenas chama funções como notas.minhas() ou respostas.avaliar().

7.2 APIs REST

A API segue o estilo REST, com recursos identificados por URL e operações expressas pelos métodos HTTP: GET para consultar, POST para criar, PUT para editar, PATCH para alterar um campo e DELETE para cancelar. Os dados trafegam em JSON, e os erros seguem um formato único, com uma mensagem legível. A documentação interativa é gerada pelo Swagger. O Quadro 11 apresenta os principais endpoints e o caso de uso atendido.

Quadro 11 – Principais endpoints da API REST

Método e rota	**Perfil**	**Função**
POST /api/auth/login	Público	UC01 – Acessar sistema
POST /api/auth/cadastro	Público	UC02 – Realizar cadastro
GET /api/conteudos	Público	Catálogo de cursos ativos
POST, PUT /api/conteudos	Secretaria	UC07 e UC08 – Adicionar e editar conteúdo
PATCH /api/conteudos/{id}/ativo	Secretaria	UC09 – Desativar conteúdo
GET /api/atividades/minhas	Aluno	UC03 – Atividades e situação de cada uma
POST /api/respostas	Aluno	UC04 – Realizar avaliação
GET /api/notas/minhas	Aluno	UC05 – Acessar notas
POST /api/respostas/{id}/nota	Secretaria	UC06 – Avaliar atividade
POST /api/usuarios	Administrador	UC10 – Cadastrar secretaria
PATCH /api/usuarios/{id}/ativo	Administrador	UC11 – Desativar cadastro

Método e rota	Perfil	Função
GET /api/relatorios/resumo	Administrador	Indicadores da plataforma
GET /api/certificados/validar/{codigo}	Público	Validação de certificado
GET /api/sincronizacao	Aluno	Pacote de dados para o aplicativo offline

Fonte: elaborado pelos autores (2026).

7.3 Autenticação

No login, a API confere a senha com o algoritmo BCrypt e bloqueia usuários inativos. Em seguida, gera um token do tipo Bearer, criptografado pelo próprio ASP.NET Core (MICROSOFT, 2026), com validade de oito horas, contendo o identificador, o nome, o e-mail e o perfil do usuário. O site guarda esse token e o envia em todas as requisições, no cabeçalho Authorization. Cada rota declara quais perfis podem acessá-la e, a cada requisição, o usuário é buscado no banco: se o administrador o desativou, o token deixa de valer imediatamente.

7.4 Segurança

Quadro 12 – Medidas de segurança

Ameaça	Proteção adotada
Vazamento de senha	Hash BCrypt com salt; a senha nunca é armazenada em texto
Descoberta de e-mails cadastrados	Mesma mensagem para e-mail inexistente e senha incorreta
Acesso indevido	Token Bearer, autorização por perfil em cada rota e nova verificação do perfil no serviço
SQL Injection	Todos os comandos SQL utilizam parâmetros
Dados inválidos	Validação no domínio (C#) e no banco (CHECK, UNIQUE e triggers)
Chamadas de outros sites	CORS liberado apenas para as origens configuradas
Exposição de detalhes internos	Middleware que devolve mensagem genérica em erros inesperados e registra o detalhe em log
Rastreabilidade	Log de auditoria preenchido por triggers

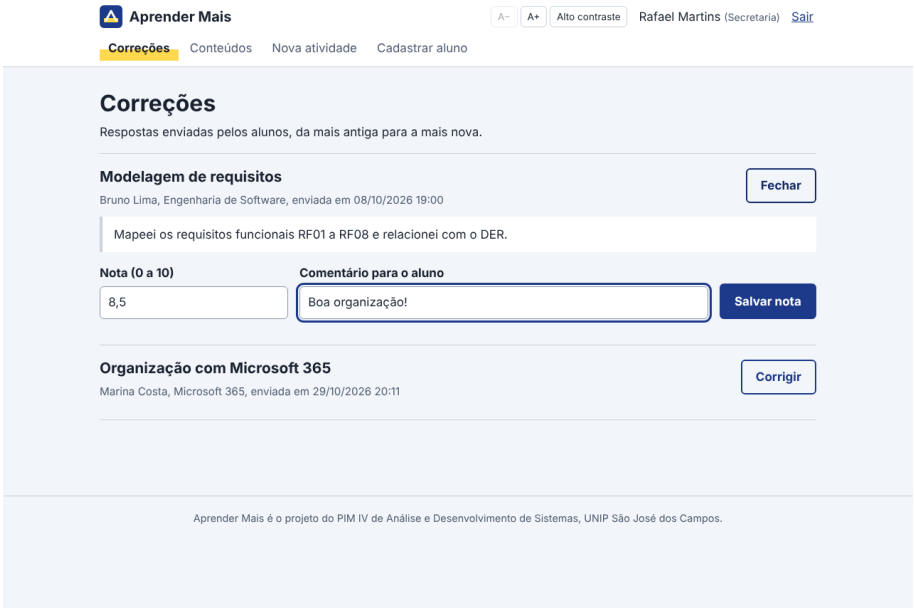
Fonte: elaborado pelos autores (2026).

7.5 Estrutura administrativa e integração entre módulos

A área da secretaria oferece a fila de correções, com nota e feedback; a gestão completa de conteúdos, com ativação e desativação; a criação de atividades com prazo; e o cadastro de alunos. A área do administrador apresenta um painel com indicadores e desempenho por curso, gerados pelas procedures de relatório, além da

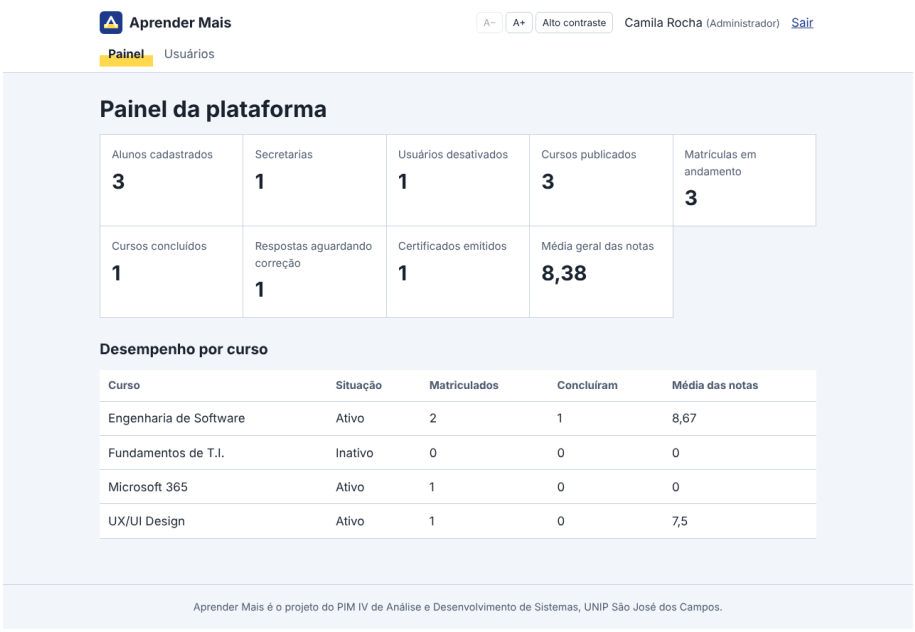
lista de usuários com busca, cadastro de secretarias e ativação ou desativação de contas.

Figura 6 – Fila de correções da secretaria



Fonte: elaborado pelos autores (2026).

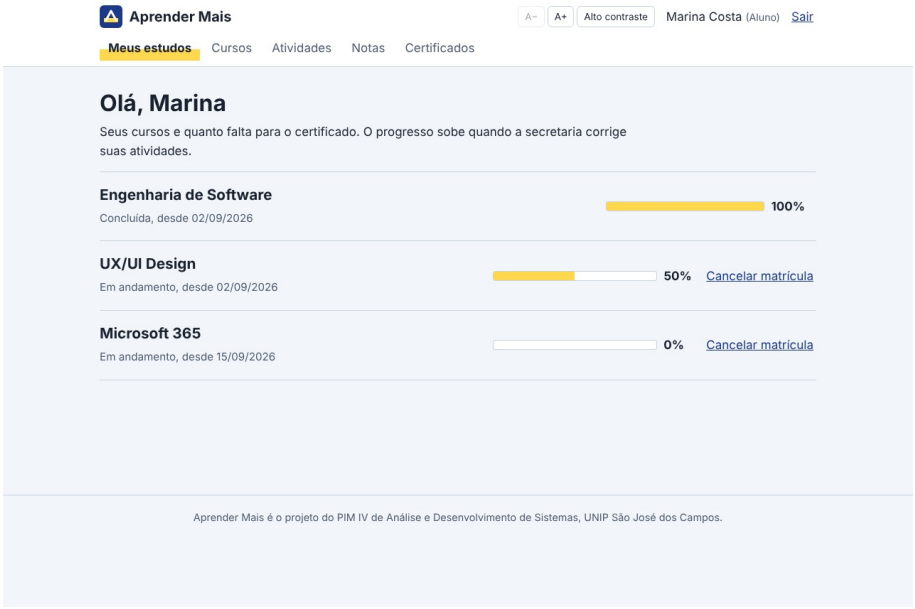
Figura 7 – Painel do administrador com indicadores



Fonte: elaborado pelos autores (2026).

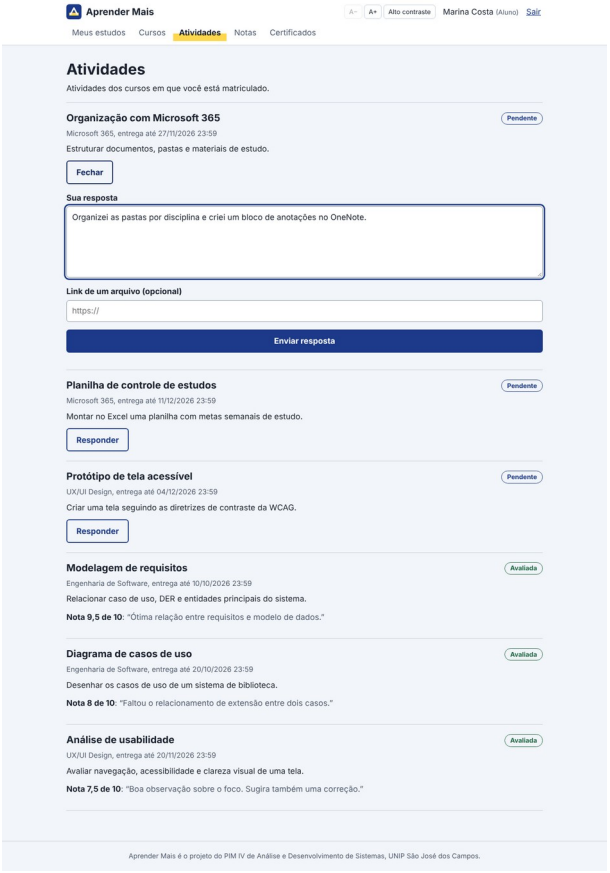
A integração entre os módulos acontece em cadeia. Quando a secretaria salva uma nota, a API grava a nota, recalcula o progresso pela procedure sp_calcular_progresso e, se o aluno atingiu 100%, emite o certificado pela procedure sp_emitir_certificado. O resultado aparece imediatamente na área do aluno no site e, na próxima sincronização, no aplicativo. As Figuras 8 e 9 mostram a área do aluno.

Figura 8 – Área do aluno com o progresso dos cursos



Fonte: elaborado pelos autores (2026).

Figura 9 – Atividades do aluno com envio de resposta



Fonte: elaborado pelos autores (2026).

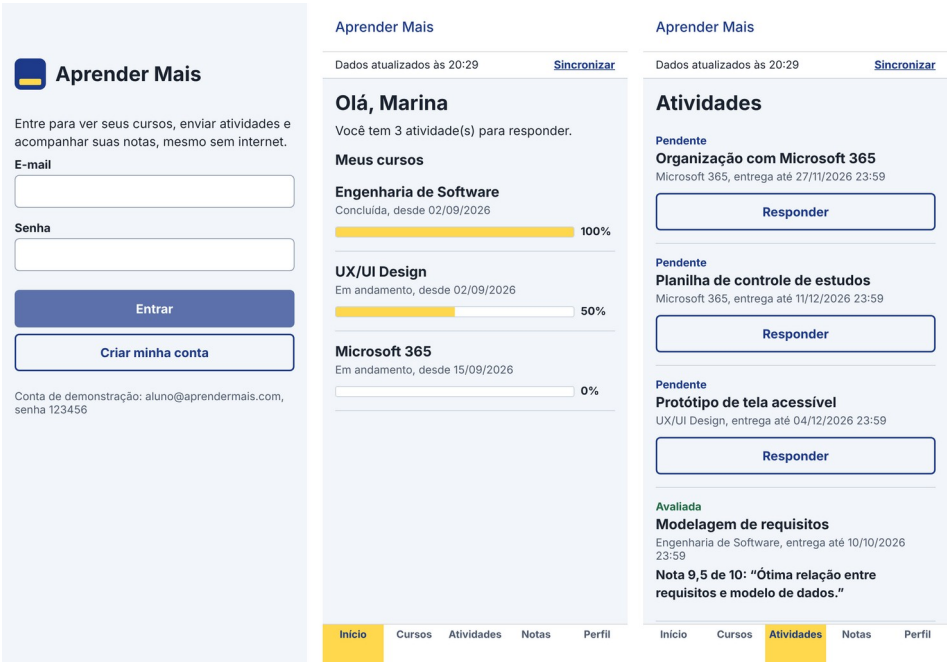
8 DESENVOLVIMENTO DA SOLUÇÃO MOBILE

O aplicativo, relacionado à disciplina de Desenvolvimento Mobile, é voltado aos alunos e foi desenvolvido em React Native com a plataforma Expo, utilizando JavaScript (EXPO, 2026). Ele funciona em Android e iOS, pelo aplicativo Expo Go, e também no navegador, o que facilita os testes. A secretaria e o administrador utilizam o site ou o desktop.

8.1 Telas principais

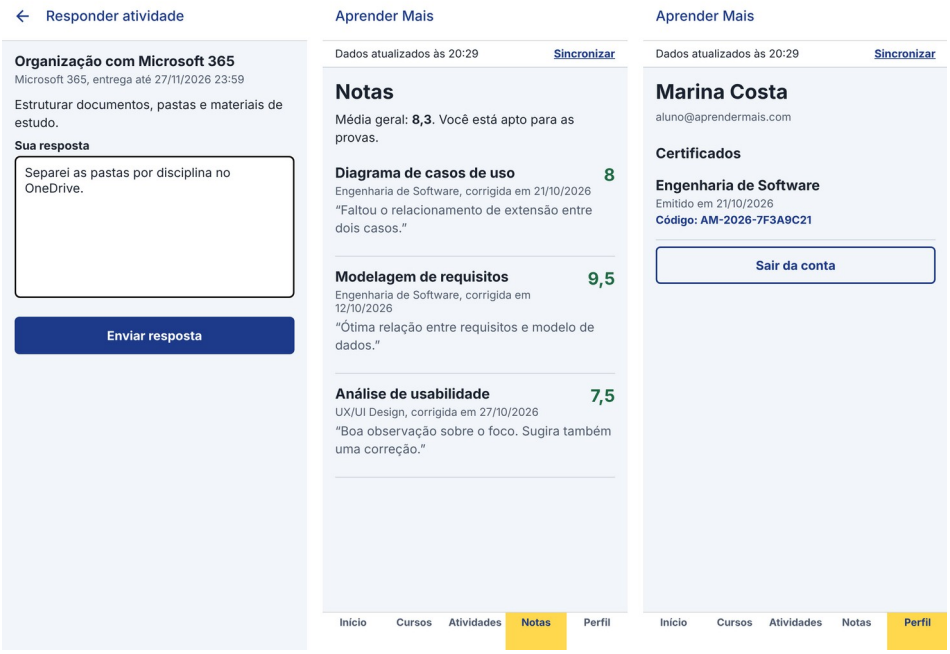
O aplicativo possui as telas de login e cadastro; a tela inicial, com as matrículas e o progresso de cada curso; o catálogo de cursos, com matrícula; a lista de atividades, com a situação de cada entrega, nota e feedback; a tela de resposta; o boletim de notas, com a média geral; e o perfil, com os certificados e a opção de sair. As Figuras 10 e 11 apresentam as telas.

Figura 10 – Telas de login, início e atividades do aplicativo



Fonte: elaborado pelos autores (2026).

Figura 11 – Telas de resposta, notas e perfil do aplicativo

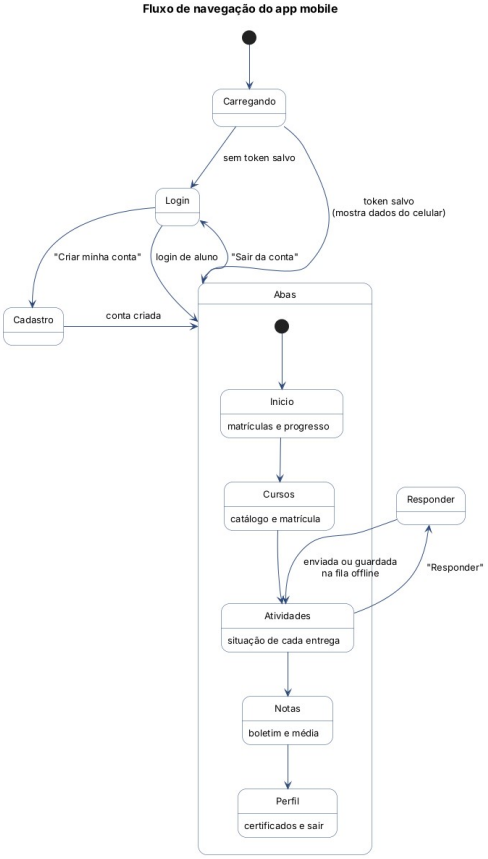


Fonte: elaborado pelos autores (2026).

8.2 Fluxo de navegação e autenticação

A navegação utiliza a biblioteca React Navigation, com uma pilha de telas para login, cadastro e resposta e abas inferiores para início, cursos, atividades, notas e perfil (Figura 12). Sem login, o aplicativo mostra apenas as telas de login e cadastro. A autenticação usa o mesmo endpoint do site: o token recebido é salvo no armazenamento permanente do celular (AsyncStorage), de modo que o aluno continua conectado ao reabrir o aplicativo. Se o token expirar, os dados locais são apagados e o aluno volta ao login.

Figura 12 – Fluxo de navegação do aplicativo

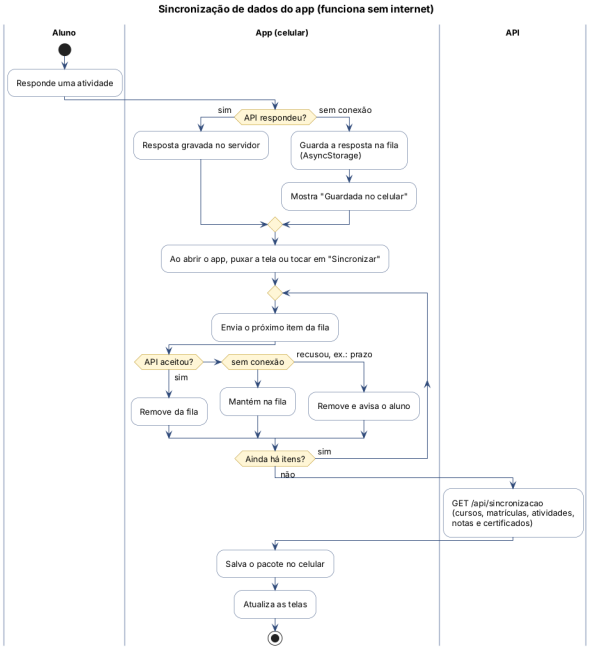


Fonte: elaborado pelos autores (2026).

8.3 Sincronização de dados

O principal diferencial do aplicativo é funcionar sem internet. Ao abrir, ele mostra imediatamente os dados salvos no celular e, em seguida, sincroniza com a API. Quando o aluno responde uma atividade sem conexão, a resposta é guardada em uma fila local e a atividade passa a aparecer como "Guardada no celular". Na próxima sincronização, disparada ao abrir o aplicativo, ao arrastar a tela para baixo ou ao tocar em "Sincronizar", a fila é enviada e o pacote de dados é baixado novamente. Se a API recusar algum item da fila, por exemplo por prazo vencido, ele é removido e o aluno recebe um aviso (Figuras 13 e 14).

Figura 13 – Diagrama de atividade da sincronização de dados



Fonte: elaborado pelos autores (2026).

Figura 14 – Resposta guardada sem internet, aviso de conexão e situação após a sincronização



Fonte: elaborado pelos autores (2026).

8.4 Integração com APIs

O aplicativo consome os mesmos endpoints do site para login, cadastro, matrícula e envio de respostas. Para a sincronização, foi criado o endpoint GET /api/sincronizacao, que devolve em uma única requisição os cursos, as matrículas, as

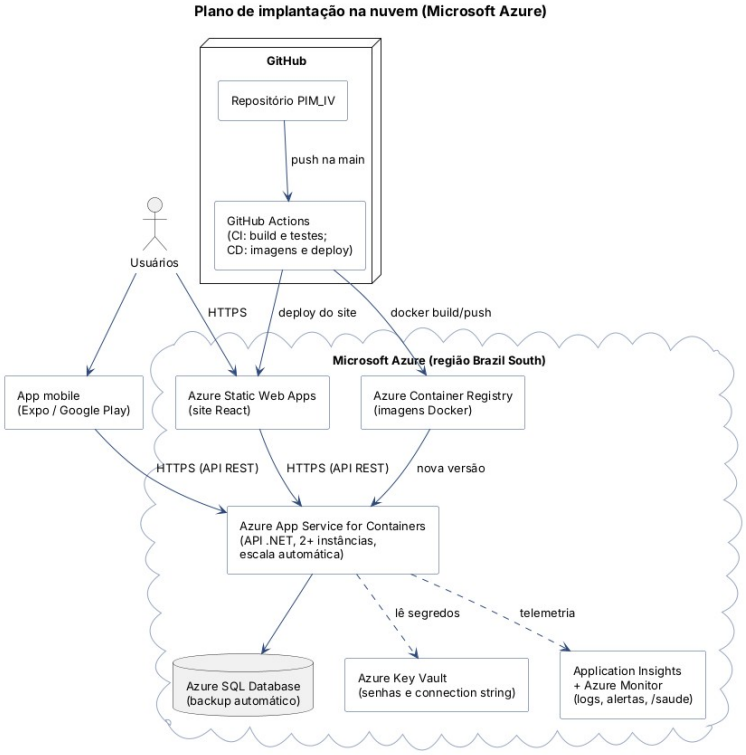
atividades, as notas e os certificados do aluno. Com isso, o aplicativo economiza bateria e dados móveis e tem tudo o que precisa para funcionar offline.

9 INFRAESTRUTURA EM NUVEM E DEVOPS

Esta seção, relacionada à disciplina de Cloud Computing e DevOps, apresenta o plano de execução para implantar a solução na Microsoft Azure. Segundo Kim et al. (2018), as práticas de DevOps buscam entregas frequentes e confiáveis por meio de automação, integração contínua e monitoramento. A parte de contêineres e de integração contínua já foi implementada no repositório; a implantação em nuvem é o plano descrito a seguir.

9.1 Arquitetura em nuvem e serviços utilizados

Figura 15 – Plano de implantação na Microsoft Azure



Fonte: elaborado pelos autores (2026).

Quadro 13 – Serviços de nuvem utilizados

Serviço	Função na solução
Azure App Service for Containers	Executa a API .NET a partir da imagem Docker, com HTTPS e escala automática
Azure SQL Database	Banco de dados gerenciado, com backup automático e alta disponibilidade
Azure Static Web Apps	Hospeda o site React com CDN e HTTPS, no plano gratuito
Azure Container Registry	Armazena as imagens Docker geradas no pipeline
Azure Key Vault	Guarda a connection string e as senhas fora do código
Application Insights e Azure Monitor	Logs, métricas, alertas e testes de disponibilidade

Serviço	Função na solução
Expo EAS e Google Play	Geração e distribuição do aplicativo

Fonte: elaborado pelos autores (2026).

A região sugerida é Brazil South (São Paulo), pela menor latência para os alunos.

9.2 Contêineres e pipeline CI/CD

A API e o site possuem arquivos Dockerfile com construção em duas etapas: a primeira compila o projeto e a segunda gera uma imagem menor, apenas com o necessário para executá-lo. O arquivo docker-compose.yml sobe o SQL Server, executa os scripts do banco e inicia a API e o site com um único comando. A integração contínua foi implementada com GitHub Actions: a cada envio de código para o ramo principal, o pipeline compila a API e executa os 49 testes, compila o desktop em um ambiente Windows, verifica e compila o site, gera o pacote do aplicativo e, se tudo passar, constrói as imagens Docker. A entrega contínua planejada publica as imagens no Container Registry, implanta o site no Static Web Apps e a API em um ambiente de homologação, testa o endpoint de saúde e só então promove a nova versão para produção, sem tempo fora do ar.

9.3 Monitoramento, escalabilidade e segurança

O monitoramento utiliza o endpoint /saude, consultado pelo Docker, pelo App Service e pelos testes de disponibilidade; os erros inesperados são registrados em log e enviados ao Application Insights, com alertas por e-mail para taxa de erros, indisponibilidade e uso alto de CPU. A escalabilidade é possível porque a API não guarda estado entre requisições: o App Service pode executar de duas a cinco instâncias conforme a carga, o banco pode mudar de plano sem alteração no código e o site é distribuído por CDN. Para várias instâncias, as chaves de criptografia dos tokens devem ficar em local compartilhado, como o Azure Blob Storage protegido pelo Key Vault. Na segurança, adota-se HTTPS em todos os serviços, firewall no banco, segredos no Key Vault acessados por identidade gerenciada, usuário de banco próprio para a API e ramo principal protegido, com revisão obrigatória de código.

10 GERENCIAMENTO ÁGIL DO PROJETO

O projeto foi gerenciado com Scrum, que organiza o trabalho em ciclos curtos chamados sprints, com papéis, eventos e artefatos definidos (SCHWABER; SUTHERLAND, 2020), complementado por um quadro Kanban para visualizar o andamento das tarefas. Foram planejadas doze sprints de uma semana, de 1º de setembro a 23 de novembro de 2026. O Quadro 14 apresenta os papéis do time.

Quadro 14 – Time Scrum

Integrante	Papel	Foco técnico
Gabriel Souza Maciel	Scrum Master	Front-end web e mobile
Julia Peneluppi de Faria	Desenvolvimento	Experiência do usuário, documentação e banco de dados
Vitor Gabriel Pereira Geraldo	Product Owner	Back-end em C# e desktop

Fonte: elaborado pelos autores (2026).

Os eventos ocorreram da seguinte forma: planejamento da sprint às terças-feiras, reunião diária de quinze minutos por mensagem e revisão e retrospectiva às segundas-feiras. A definição de pronto exigiu código no GitHub com revisão, integração contínua aprovada, funcionalidade testada pelo Product Owner e documentação atualizada.

10.1 Product Backlog

O Product Backlog reúne as histórias de usuário US01 a US12, herdadas do PIM III, as histórias US13 a US19 e os itens técnicos T01 a T05, incluídos no PIM IV. O total é de 128 pontos, com velocidade média planejada de cerca de onze pontos por sprint. O Quadro 15 apresenta um resumo.

Quadro 15 – Product Backlog (resumo)

ID	História de usuário ou item técnico	Prioridade	Pontos
US01–US05	Como aluno, quero me cadastrar, acessar o sistema, os conteúdos e as avaliações e ver minhas notas	Alta	29
US06–US10	Como secretaria, quero gerenciar acessos, avaliar atividades e adicionar, editar e desativar conteúdos	Média	26
US11–US12	Como administrador, quero cadastrar secretarias e desativar cadastros	Baixa	8
US13	Como aluno, quero usar o aplicativo mesmo sem internet	Alta	8
US14–US15	Certificado automático e validação pública do certificado	Alta / Média	8
US16–US18	Relatórios, acessibilidade e criação de atividades com	Média / Alta	13

ID	História de usuário ou item técnico	Prioridade	Pontos
	prazo		
US19	Como secretaria ou administrador, quero usar o sistema no computador	Média	8
T01–T05	Banco de dados, API, testes, Docker e CI, documento e apresentação	Alta / Média	34

Fonte: elaborado pelos autores (2026).

10.2 Sprint Backlog e cronograma das sprints

Cada sprint teve um objetivo e um Sprint Backlog com as tarefas e os responsáveis. O Quadro 16 apresenta o cronograma e a situação das sprints na data de elaboração deste documento.

Quadro 16 – Cronograma das sprints

Sprint	Período	Objetivo	Situação
S1	01/09 a 07/09	Kick-off, revisão do PIM III, caracterização, modelo de negócio e Product Backlog	Concluída
S2	08/09 a 14/09	Arquitetura em camadas e migração do banco para SQL Server	Concluída
S3	15/09 a 21/09	Domínio e aplicação em C#, com testes automatizados	Concluída
S4	22/09 a 28/09	Procedures, triggers, views, dados iniciais e repositórios SQL	Concluída
S5	29/09 a 05/10	API REST, autenticação, Swagger, atividades, notas e certificados	Concluída
S6	06/10 a 12/10	Site: login, cadastro e área do aluno	Concluída
S7	13/10 a 19/10	Site: secretaria, administração, relatórios e acessibilidade	Concluída
S8	20/10 a 26/10	Aplicativo: telas, navegação e sincronização offline	Concluída
S9	27/10 a 02/11	Desktop, diagramas de atividade, Docker e integração contínua	Concluída
S10	03/11 a 09/11	Testes integrados com SQL Server e homologação com usuários	Concluída
S11	10/11 a 12/11	Documento final em ABNT	Em andamento
S12	17/11 a 23/11	Revisão, apresentação e entrega	Planejada

Fonte: elaborado pelos autores (2026).

Como exemplo de Sprint Backlog, a Sprint 8 teve como objetivo entregar o aplicativo do aluno, com as tarefas: criar o projeto Expo e a navegação em pilha e abas; desenvolver as oito telas; criar o endpoint de sincronização na API; e implementar o armazenamento local e a fila offline de respostas, atendendo à história US13.

10.3 Quadro Kanban e definição das entregas

O quadro Kanban foi organizado nas colunas "A fazer", "Fazendo", "Em revisão" e "Concluído". Na Sprint 11, as histórias US01 a US19 e os itens técnicos T01 a T04 estavam concluídos, assim como os testes integrados com SQL Server e a homologação; o documento final estava em revisão; e a apresentação e a entrega oficial estavam a fazer. O Quadro 17 apresenta as entregas definidas.

Quadro 17 – Definição das entregas

Entrega	Conteúdo	Sprint	Critério de aceite
E1 – Planejamento	Etapas 1 e 2 e Product Backlog	S1	Aprovação do Product Owner
E2 – Banco de dados	Scripts e script completo	S4	Execução sem erros em banco vazio
E3 – API	API REST documentada no Swagger	S5	Endpoints respondendo e testes aprovados
E4 – Web	Site com os três perfis	S7	Fluxo completo do aluno ao certificado
E5 – Mobile	Aplicativo com modo offline	S8	Resposta offline enviada após sincronizar
E6 – Desktop e DevOps	Windows Forms, Docker e CI	S9	Pipeline aprovado no GitHub
E7 – Documento	PIM em ABNT	S11	Checklist ABNT do manual atendido
E8 – Entrega oficial	Postagem e apresentação	S12	Postado dentro do prazo

Fonte: elaborado pelos autores (2026).

11 CONCLUSÃO

Este trabalho teve como objetivo desenvolver e planejar a implantação da solução tecnológica integrada da escola Aprender Mais, dando continuidade à modelagem realizada no PIM III. O objetivo foi alcançado: a organização passou a contar com um sistema único, acessível pelo site, pelo aplicativo e pelo computador, que substitui as planilhas e os e-mails por processos automatizados de matrícula, entrega, correção, cálculo de progresso e emissão de certificados validáveis.

A arquitetura em camadas, aliada aos princípios da orientação a objetos, mostrou-se decisiva para que as três versões compartilhassem exatamente as mesmas regras de negócio, sem duplicação de código. O banco de dados em SQL Server, com restrições, procedures, triggers e views, garantiu a integridade das informações mesmo fora da aplicação. A API REST, com autenticação por token e autorização por perfil, integrou as interfaces de forma segura e padronizada.

Entre os resultados de maior impacto para o público da escola, destacam-se o aplicativo que funciona sem internet, os recursos de acessibilidade para pessoas com baixa visão e o certificado com validação pública, que conectam a solução aos princípios de inclusão e cidadania. No aspecto empreendedor, a análise de viabilidade indicou custos de infraestrutura baixos em relação à receita esperada.

A adoção do Scrum, com sprints semanais, Kanban e integração contínua, permitiu entregas incrementais e verificáveis e uma divisão clara de responsabilidades no time. Como próximos passos, recomenda-se executar a implantação planejada na Microsoft Azure, realizar testes com usuários reais, publicar o aplicativo na loja e ampliar o catálogo de cursos com conteúdos que valorizem a diversidade.

Conclui-se que o projeto integrou os conteúdos das disciplinas do quarto semestre (empreendedorismo, desenvolvimento web e mobile, programação em .NET, banco de dados, computação em nuvem, DevOps, gestão ágil e relações étnico-raciais) em um produto funcional, alinhado às necessidades da organização e à sua transformação digital.

REFERÊNCIAS

BRASIL. **Lei nº 10.639, de 9 de janeiro de 2003**. Altera a Lei nº 9.394, de 20 de dezembro de 1996, para incluir no currículo oficial da rede de ensino a obrigatoriedade da temática "História e Cultura Afro-Brasileira". Brasília, DF: Presidência da República, 2003.

BRASIL. **Lei nº 13.146, de 6 de julho de 2015**. Institui a Lei Brasileira de Inclusão da Pessoa com Deficiência (Estatuto da Pessoa com Deficiência). Brasília, DF: Presidência da República, 2015.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018**. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018.

DORNELAS, José. **Empreendedorismo**: transformando ideias em negócios. 8. ed. São Paulo: Empreende, 2021.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

EXPO. **Expo documentation**. [S. l.], 2026. Disponível em: https://docs.expo.dev/. Acesso em: 29 out. 2026.

FOWLER, Martin. **Padrões de arquitetura de aplicações corporativas**. Porto Alegre: Bookman, 2006.

KIM, Gene *et al*. **Manual de DevOps**: como obter agilidade, confiabilidade e segurança em organizações tecnológicas. Rio de Janeiro: Alta Books, 2018.

LARMAN, Craig. **Utilizando UML e padrões**: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo. 3. ed. Porto Alegre: Bookman, 2007.

MARTIN, Robert C. **Arquitetura limpa**: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

MICROSOFT. **Documentação do ASP.NET Core e do Azure**. [S. l.], 2026. Disponível em: https://learn.microsoft.com/pt-br/. Acesso em: 29 out. 2026.

OSTERWALDER, Alexander; PIGNEUR, Yves. **Business Model Generation**: inovação em modelos de negócios. Rio de Janeiro: Alta Books, 2011.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

REACT. **React documentation**. [S. l.], 2026. Disponível em: https://react.dev/. Acesso em: 29 out. 2026.

SCHWABER, Ken; SUTHERLAND, Jeff. **O Guia do Scrum**: o guia definitivo para o Scrum: as regras do jogo. [S. l.], 2020. Disponível em: https://scrumguides.org/. Acesso em: 29 out. 2026.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson, 2018.

W3C. **Web Content Accessibility Guidelines (WCAG) 2.1**. [S. l.], 2018. Disponível em: https://www.w3.org/TR/WCAG21/. Acesso em: 29 out. 2026.

APÊNDICE A – REPOSITÓRIO E ACESSO AO SISTEMA

O código-fonte completo, os scripts do banco de dados, os diagramas e a documentação de cada etapa estão disponíveis no repositório público https://github.com/souzaa7/PIM_IV. O Quadro 18 apresenta as contas de demonstração, todas com a senha 123456.

Quadro 18 – Contas de demonstração

Perfil	**E-mail**	**Onde utilizar**
Aluno	aluno@aprendermais.com	Site, aplicativo e desktop
Aluno	bruno@aprendermais.com	Site, aplicativo e desktop
Secretaria	secretaria@aprendermais.com	Site e desktop
Administrador	admin@aprendermais.com	Site e desktop
Aluno inativo	ana@aprendermais.com	Teste do bloqueio de acesso

Fonte: elaborado pelos autores (2026).